

“Año de la Esperanza y el Fortalecimiento de la Democracia”

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA



CURSO:

METODOS NUMERICOS

GRUPO:

G3 MARTES: 8:00 - 10:00

DOCENTE:

MIRANDA FERNANDEZ, JOSUE ALFONSO

INTEGRANTE:

BÉDON AVILES, JAIR ANTONIO 24190047

Lima-Perú

2026

Sistema Portátil de Monitoreo Fisiológico

Análisis Numérico de Impedancia Bioeléctrica

Contexto y Datos del Experimento

Se dispone de **30 mediciones** de magnitud de impedancia bioeléctrica $|Z|$ (Ω) en función de la frecuencia de excitación f (Hz), tomadas a temperatura constante de 37 °C sobre tejido simulado. Los datos cubren el rango $f \in [100, 2730]$ Hz.

Tabla 1. Datos experimentales completos — 30 pares (f , $|Z|$)

N°	f (Hz)	Z (Ω)	N°	f (Hz)	Z (Ω)
1	100	152.3	16	810	131
2	120	149.1	17	895	131.3
3	145	146.8	18	985	131.9
4	170	144.9	19	1080	132.7
5	200	142	20	1180	133.8
6	235	139.5	21	1290	135.2
7	270	137.9	22	1410	136.9
8	310	136.1	23	1540	138.9
9	355	134.8	24	1680	141.1
10	405	133.6	25	1830	143.5
11	460	132.7	26	1990	146.1
12	520	131.9	27	2160	149
13	585	131.4	28	2340	152.2
14	655	131.1	29	2530	155.6
15	730	130.9	30	2730	159.2

Parte A — Análisis Exploratorio

A.1 Gráfica de $|Z|(f)$

La figura siguiente se genera ejecutando el Código A.1 en MATLAB. El gráfico muestra la magnitud de impedancia $|Z|$ versus frecuencia f con marcadores en cada punto experimental.

Código A.1 — MATLAB: Gráfica exploratoria

% =====
% PARTE A – Análisis exploratorio de Z (f)
% =====
% Datos experimentales
f = [100, 120, 145, 170, 200, 235, 270, 310, 355, 405, ...
460, 520, 585, 655, 730, 810, 895, 985, 1080, 1180, ...
1290, 1410, 1540, 1680, 1830, 1990, 2160, 2340, 2530, 2730];
Z = [152.3, 149.1, 146.8, 144.9, 142.0, 139.5, 137.9, 136.1, ...
134.8, 133.6, 132.7, 131.9, 131.4, 131.1, 130.9, 131.0, ...
131.3, 131.9, 132.7, 133.8, 135.2, 136.9, 138.9, 141.1, ...
143.5, 146.1, 149.0, 152.2, 155.6, 159.2];
% ---- Gráfica ----
figure('Name','Parte A - Análisis exploratorio','NumberTitle','off');
plot(f, Z, 'bo-', 'LineWidth', 1.5, 'MarkerFaceColor', 'b', 'MarkerSize', 5);
grid on;
xlabel('Frecuencia f (Hz)', 'FontSize', 13);
ylabel(' Z (\Omega)', 'FontSize', 13);
title('Magnitud de impedancia bioeléctrica vs. frecuencia', 'FontSize', 14);
% Marcar mínimo visual
[Zmin, idx_min] = min(Z);
hold on;
plot(f(idx_min), Zmin, 'r*', 'MarkerSize', 14, 'LineWidth', 2);
text(f(idx_min)+50, Zmin+0.5, ...
sprintf('Mínimo \approx (%.0f Hz, %.1f \Omega)', f(idx_min), Zmin), ...
'FontSize', 11, 'Color', 'r');
legend('Datos experimentales', 'Mínimo local', 'Location', 'northeast');
% Estadísticas básicas
fprintf('Frecuencia de mínimo: %.0f Hz\n', f(idx_min));
fprintf('Impedancia mínima: %.1f Ohm\n', Zmin);
fprintf('Impedancia máxima: %.1f Ohm (en f=%.0f Hz)\n', max(Z), f(Z==max(Z)));

A.2 Discusión de la tendencia observada

Forma general de la curva. La curva $|Z|(f)$ presenta un mínimo pronunciado aproximadamente en $f \approx 730 \text{ Hz}$ (punto 15), donde $|Z|_{\min} = 130.9 \Omega$. A ambos lados del mínimo la impedancia crece, formando una curva con forma de **U o parábola asimétrica**.

Interpretación física (modelo Cole-Cole simplificado). En bioimpedancia, el tejido biológico se modela eléctricamente como una resistencia extracelular R_e en paralelo con la combinación serie de una resistencia intracelular R_i y una capacitancia de membrana C_m . La función de transferencia resultante es:

$$|Z(j\omega)| = |R_e \cdot (R_i + 1/(j\omega C_m)) / (R_e + R_i + 1/(j\omega C_m))|$$

A bajas frecuencias: la corriente no penetra la membrana celular ($1/\omega C_m \rightarrow \infty$), dominando la vía extracelular. La impedancia es alta y tiende a R_e .

A frecuencias intermedias: la capacitancia de membrana comienza a conducir; la corriente penetra al intracelular. La impedancia total disminuye hasta el mínimo en la frecuencia característica

$$f_c = 1/(2\pi\sqrt{(R_i R_e C_m)}).$$

A altas frecuencias: $1/\omega C_m \rightarrow 0$, ambas resistencias quedan en paralelo. Sin embargo, efectos inductivos parásitos del sistema de medición elevan la impedancia medida por encima del mínimo.

Mínimo local. Inspeccionando la tabla: el punto **15 (f = 730 Hz, |Z| = 130.9 Ω)** es el mínimo global en el rango medido. Los puntos vecinos confirman el descenso suave previo y la subida posterior. Se estima la frecuencia del mínimo en $f_{min} \approx 730$ Hz.

Parte B — Interpolación

B1 — Interpolación Polinómica

B1.a Método Matricial (Vandermonde)

Dados $n = 30$ puntos $(f_i, |Z|_i), i = 1, \dots, 30$, se busca el polinomio de grado $n-1 = 29$ de la forma:

$$p(f) = a_0 + a_1 f + a_2 f^2 + \dots + a_{29} f^{29}$$

Condicionando el paso de p por los 30 puntos se obtiene el sistema lineal:

$$V \cdot a = z$$

donde V es la matriz de Vandermonde (30×30):

$$V_{ij} = f_i^{j-1}, \quad i = 1, \dots, 30; \quad j = 1, \dots, 30$$

el vector de incógnitas es $a = [a_0, a_1, \dots, a_{29}]^T$ y $z = [|Z|_1, \dots, |Z|_{30}]^T$.

La solución es $a = V \backslash z$ (operador de división izquierda de MATLAB, equivalente a $a = V^{-1} \cdot z$).

Código B.1a — MATLAB: Interpolación matricial (Vandermonde) y de Lagrange

% =====
% PARTE B1 – Interpolación polinómica
% Método Matricial (Vandermonde) + Lagrange
% =====
clear; clc;
f = [100, 120, 145, 170, 200, 235, 270, 310, 355, 405, ...
460, 520, 585, 655, 730, 810, 895, 985, 1080, 1180, ...
1290, 1410, 1540, 1680, 1830, 1990, 2160, 2340, 2530, 2730];
z = [152.3, 149.1, 146.8, 144.9, 142.0, 139.5, 137.9, 136.1, ...
134.8, 133.6, 132.7, 131.9, 131.4, 131.1, 130.9, 131.0, ...
131.3, 131.9, 132.7, 133.8, 135.2, 136.9, 138.9, 141.1, ...
143.5, 146.1, 149.0, 152.2, 155.6, 159.2];
n = length(f);

```

f_eval = 1000;           % Punto de evaluación
f_fine = linspace(min(f), max(f), 2000); % Malla fina

% -----
% 1. MÉTODO MATRICIAL - Vandermonde grado 29
% -----
% Construir matriz de Vandermonde normalizada
% (normalizar mejora el condicionamiento)
f_mean = mean(f);
f_std = std(f);
f_norm = (f - f_mean) / f_std; % Variables normalizadas

V = zeros(n, n);
for j = 1:n
    V(:, j) = f_norm(:).^ (j-1); % Columna j: f_norm^(j-1)
end

% Resolver sistema lineal
a = V \ Z(:); % Coeficientes en espacio normalizado

% Evaluar polinomio en malla fina y en f=1000 Hz
f_fine_norm = (f_fine - f_mean) / f_std;
f_eval_norm = (f_eval - f_mean) / f_std;

p_fine = zeros(size(f_fine));
for j = 1:n
    p_fine = p_fine + a(j) * f_fine_norm.^ (j-1);
end

p_eval = 0;
for j = 1:n
    p_eval = p_eval + a(j) * f_eval_norm.^ (j-1);
end
fprintf('Vandermonde |Z|(1000 Hz) = %.4f Ohm\n', p_eval);

% -----
% 2. INTERPOLACIÓN DE LAGRANGE
% -----
% Función auxiliar: evalúa el polinomio de Lagrange
% en un punto x dado los nodos xn y valores yn
lagrange = @(xn, yn, x) sum(arrayfun(@(k) ...
    yn(k) * prod((x - xn([1:k-1, k+1:end]))) ./ ...
    (xn(k) - xn([1:k-1, k+1:end]))), ...
    1:length(xn)));

% Evaluar en f = 1000 Hz
L_eval = lagrange(f, Z, f_eval);
fprintf('Lagrange |Z|(1000 Hz) = %.4f Ohm\n', L_eval);

% Evaluar en malla fina (puede ser lento para n=30)
L_fine = arrayfun(@(xi) lagrange(f, Z, xi), f_fine);

```

% _____
% 3. GRÁFICA comparativa grado 29
% _____
figure('Name','B1a - Vandermonde vs Lagrange grado 29');
plot(f, Z, 'ko', 'MarkerFaceColor','k','MarkerSize',5); hold on;
plot(f_fine, p_fine, 'b-', 'LineWidth',1.5);
plot(f_fine, L_fine, 'r--','LineWidth',1.5);
xline(f_eval,'g--','LineWidth',1.2);
ylim([100 250]);
legend('Datos','Vandermonde gr.29','Lagrange gr.29','f=1000 Hz','Location','north');
xlabel('f (Hz)'); ylabel(' Z (\Omega)');
title('Interpolación polinómica de grado 29 - Ambos métodos');
grid on;

B1.b Oscilaciones de Runge — Grado 29 vs. polinomios escalonados

Problema con el polinomio global de alto grado. El fenómeno de Runge establece que, al aumentar el grado del polinomio interpolador con nodos equiespaciados (o aproximadamente equiespaciados), el error puede crecer exponencialmente en las regiones cercanas a los extremos del intervalo. Para un polinomio de grado 29, el número de condición de la matriz de Vandermonde es extremadamente alto ($\kappa(V) \sim 10^{30}$), lo que hace la solución numéricamente inestable.

Adicionalmente, dado que los datos de impedancia tienen forma de curva suave con un mínimo, un polinomio de alto grado intentará forzar $n-1$ oscilaciones, generando valores espurios entre los puntos de datos.

Estrategia de polinomios escalonados (piecewise). Se divide el dominio de frecuencias en subintervalos y se ajusta un polinomio de menor grado (5, 10 o 15) en cada segmento, lo que reduce drásticamente las oscilaciones y el número de condición.

Código B.1b — MATLAB: Comparación Runge — grado 29 vs. escalonados

% =====
% PARTE B1b — Fenómeno de Runge: grado 29 vs. escalonados
% =====
% (f y Z definidos como en Código B.1a)
grados = [5, 10, 15, 29];
colores = {'g','m','c','r'};
figure('Name','B1b - Fenómeno de Runge');
plot(f, Z, 'ko','MarkerFaceColor','k','MarkerSize',5,'DisplayName','Datos'); hold on;
for kg = 1:length(grados)
g = grados(kg);
% Polinomio de grado g sobre todo el intervalo
% (usar polyfit que internamente normaliza)
coef = polyfit(f, Z, g);
p_g = polyval(coef, f_fine);

plot(f_fine, p_g, [colores{kg}, '-'], 'LineWidth', 1.4, ...
'DisplayName', sprintf('Grado %d', g));
end
ylim([100 220]);
legend('Location','north','NumColumns',2);
xlabel('f (Hz)'); ylabel(' Z (\Omega)');
title('Comparación: polinomios de distintos grados – Oscilaciones de Runge');
grid on;
% Número de condición de Vandermonde para cada grado
fprintf('\nNúmero de condición de V para cada grado:\n');
for g = [5, 10, 15, 29]
V_g = zeros(n, g+1);
fn = (f - mean(f))/std(f);
for j = 0:g, V_g(:,j+1) = fn.^j; end
fprintf(' Grado %2d: cond(V) = %.2e\n', g, cond(V_g));
end

Criterio de selección. Se selecciona el polinomio de **grado 10** como el más adecuado, ya que: (a) captura la curvatura suave del mínimo sin oscilaciones visibles en el rango de datos, (b) tiene número de condición manejable, y (c) proporciona buena generalización según la validación LOO que se presenta a continuación.

B1.c Evaluación en $f = 1000$ Hz y validación Leave-One-Out (LOO)

Procedimiento LOO. La validación cruzada *leave-one-out* (LOO) con k puntos consiste en:

- Seleccionar aleatoriamente $k = 5$ puntos del conjunto de 30 como puntos de validación.
- Para cada punto $(f_i, |Z|_j)$ seleccionado:
 - (i) retirar ese punto del conjunto de datos,
 - (ii) construir el polinomio interpolador de grado 10 con los 29 puntos restantes,
 - (iii) evaluar el polinomio en f_i para obtener $p(f_i)$,
 - (iv) calcular el error relativo $e_j = |p(f_j) - |Z|_j| / |Z|_j \times 100 \%$.
- El error LOO promedio es la media de los 5 errores individuales.

Código B.1c — MATLAB: Evaluación en 1000 Hz y validación LOO

% =====
% PARTE B1c – Evaluación en f=1000 Hz y validación LOO
% =====
grado_sel = 10; % Grado seleccionado
rng(42); % Semilla para reproducibilidad
% ---- Evaluación en f = 1000 Hz ----
coef_sel = polyfit(f, Z, grado_sel);
Z_1000_poly = polyval(coef_sel, 1000);
fprintf('Polinomio grado %d – Z (1000 Hz) = %.4f Ohm\n', grado_sel, Z_1000_poly);

% ---- Validación LOO con 5 puntos aleatorios ----
k_LOO = 5;
idx_val = randperm(n, k_LOO); % Índices aleatorios
err_rel = zeros(1, k_LOO);
fprintf('\nValidación LOO (grado %d):\n', grado_sel);
fprintf('%-5s %-10s %-10s %-12s %-10s\n', ...
'Idx', 'f (Hz)', ' Z real', ' Z pred', 'Err rel (%)');
for m = 1:k_LOO
j = idx_val(m);
% Datos sin el punto j
f_loo = f([1:j-1, j+1:end]);
Z_loo = Z([1:j-1, j+1:end]);
% Interpolación sin el punto j
coef_loo = polyfit(f_loo, Z_loo, grado_sel);
Z_pred = polyval(coef_loo, f(j));
err_rel(m) = abs(Z_pred - Z(j)) / Z(j) * 100;
fprintf('%-5d %-10.0f %-10.1f %-12.4f %-10.4f\n', ...
j, f(j), Z(j), Z_pred, err_rel(m));
end
fprintf('\nError LOO promedio: %.4f %%\n', mean(err_rel));
fprintf('Error LOO máximo: %.4f %%\n', max(err_rel));

B2 — Splines Cúbicos Naturales

B2.a Construcción del spline cúbico natural

Un **spline cúbico** es una función por tramos $S(f)$ tal que en cada subintervalo $[f_i, f_{i+1}]$ es un polinomio cúbico:

$$S_i(f) = a_i + b_i(f - f_i) + c_i(f - f_i)^2 + d_i(f - f_i)^3$$

Los coeficientes $\{a_i, b_i, c_i, d_i\}$ se determinan imponiendo las condiciones:

- **Interpolación:** $S_i(f_i) = |Z|_i$ y $S_i(f_{i+1}) = |Z|_{i+1}$
- Continuidad de primera derivada: $S_i'(f_{i+1}) = S_{i+1}'(f_{i+1})$
- Continuidad de segunda derivada: $S_i''(f_{i+1}) = S_{i+1}''(f_{i+1})$
- **Condición natural (extremos libres):** $S_i''(f_i) = 0$ y $S_{n-1}''(f_n) = 0$

Para $n = 30$ puntos hay $n-1 = 29$ subintervalos y $4 \times 29 = 116$ incógnitas. Las condiciones anteriores generan exactamente 116 ecuaciones, resultando en un sistema tridiagonal que se resuelve eficientemente. La condición natural implica que fuera de los extremos el spline no fuerza curvatura artificial.

En MATLAB, la función **csape(f, Z, 'variational')** construye directamente el spline cúbico natural, y **ppval** lo evalúa en cualquier punto. Alternativamente, **spline(f, Z, f_eval)** usa condición 'not-a-knot' (ver código).

Código B.2 — MATLAB: Spline cúbico natural, comparación y evaluación

```
% =====
% PARTE B2 - Spline cúbico natural
% =====

% (f, Z, f_fine, f_eval definidos previamente)

% ---- Construir spline cúbico natural ----
% 'variational' = condición de frontera natural (S''=0 en extremos)
sp = csape(f, Z, 'variational');

% Evaluar en malla fina y en f=1000 Hz
Z_sp_fine = ppval(sp, f_fine);
Z_sp_1000 = ppval(sp, f_eval);
fprintf('Spline cúbico natural - |Z|(1000 Hz) = %.4f Ohm\n', Z_sp_1000);

% ---- Polinomio seleccionado (grado 10) para comparar ----
coef10 = polyfit(f, Z, 10);
Z_p10_fine = polyval(coef10, f_fine);
Z_p10_1000 = polyval(coef10, f_eval);

% ---- Gráfica comparativa ----
figure('Name','B2 - Spline vs Polinomio grado 10');
plot(f, Z, 'ko','MarkerFaceColor','k','MarkerSize',5,'DisplayName','Datos'); hold on;
plot(f_fine, Z_sp_fine, 'b-', 'LineWidth', 2, 'DisplayName','Spline cúbico natural');
plot(f_fine, Z_p10_fine, 'r--','LineWidth', 1.8, 'DisplayName','Polinomio grado 10');
xline(f_eval, 'g--', 'LineWidth', 1.2, 'DisplayName','f = 1000 Hz');
plot(f_eval, Z_sp_1000, 'bs', 'MarkerSize', 10, 'MarkerFaceColor','b','HandleVisibility','off');
plot(f_eval, Z_p10_1000, 'rs', 'MarkerSize', 10, 'MarkerFaceColor','r','HandleVisibility','off');
legend('Location','northwest');
xlabel('f (Hz)'); ylabel('|Z| (\Omega)');
title('Spline cúbico natural vs. Polinomio grado 10');
grid on;

% ---- Diferencia entre métodos ----
diff_1000 = abs(Z_sp_1000 - Z_p10_1000);
fprintf('Diferencia entre métodos en 1000 Hz: %.4f Ohm\n', diff_1000);
fprintf(' Spline: |Z|(1000) = %.4f Ohm\n', Z_sp_1000);
fprintf(' Polinomio grado 10: |Z|(1000) = %.4f Ohm\n', Z_p10_1000);

% ---- Residuos en los nodos para verificar interpolación ----
Z_sp_nodes = ppval(sp, f);
res_sp = max(abs(Z_sp_nodes - Z));
fprintf('\nResiduo máximo spline en nodos: %.2e Ohm\n', res_sp);
```

B2.b Comparación y discusión de confiabilidad

Resultados esperados en $f = 1000 \text{ Hz}$:

Método	$ Z (1000 \text{ Hz}) (\Omega)$	Observación
Spline cúbico natural	≈ 132.7	Coincide exactamente con dato 19
Polinomio grado 10	$\approx 132.7 \pm \delta$	Depende del condicionamiento
Lagrange grado 29	$\approx 132.7 \text{ (exacto)}$	Inestable fuera del rango

Discusión: ¿cuál método es más confiable en contexto biomédico?

- **Spline cúbico natural — PREFERIDO.** Garantiza continuidad hasta la segunda derivada, comportamiento suave entre mediciones, no genera oscilaciones espurias, y el error local no se propaga globalmente. En señales biomédicas, donde los datos son inherentemente ruidosos y la función subyacente es suave, el spline captura la forma sin artefactos.
- **Polinomio de grado 29 — NO RECOMENDADO.** Alto número de condición ($\gtrsim 10^{20}$), sensibilidad extrema a pequeñas perturbaciones en los datos, y oscilaciones de Runge en los extremos del rango.
- **Polinomio de grado 10 — Aceptable como referencia.** Buen equilibrio entre suavidad y fidelidad, útil para verificar el spline en la zona central, pero menos robusto que el spline en zonas de curvatura variable.

Conclusión: El spline cúbico natural es el método de elección para interpolación de impedancia bioeléctrica. Será empleado en las Partes C y D para derivación numérica y búsqueda de raíces.